

Flutter Feature-Based Project Structure

This document provides a detailed guide on structuring a Flutter project using a **feature-based approach**, which is scalable and maintainable for large applications.

1. Top-Level Structure

```
lib/  
├── core/           # App-wide utilities, constants, themes, and services  
├── features/       # Each feature has its own folder  
└── main.dart
```

2. `core/` Folder

Contains reusable app-wide code.

```
core/  
├── constants/      # Colors, strings, etc.  
├── utils/          # Helper functions  
├── widgets/        # Reusable widgets  
├── themes/         # AppTheme, text styles, etc.  
└── services/       # Network, storage, etc.
```

Examples:

```
core/constants/colors.dart  
core/themes/app_theme.dart  
core/utils/logger.dart  
core/widgets/custom_button.dart
```

3. `features/` Folder

Each **feature** has its own folder with all related code.

```

features/
├── auth/
│   ├── data/           # Data sources, repositories
│   ├── domain/         # Models, use cases
│   ├── presentation/   # Screens, widgets, state management
│   └── auth_feature.dart # Optional barrel file
├── home/
│   ├── data/
│   ├── domain/
│   └── presentation/
└── profile/

```

3a. presentation/ in a feature

Contains UI code.

```

presentation/
├── screens/           # Screen widgets
├── widgets/           # Feature-specific smaller widgets
└── state/             # Providers, Cubits, or Controllers

```

Example (auth/presentation):

```

screens/
  login_screen.dart
  signup_screen.dart
widgets/
  login_form.dart
state/
  auth_provider.dart

```

3b. domain/ in a feature

Contains business logic.

```

models/
  user.dart
use_cases/
  login_user.dart
  signup_user.dart

```

3c. `data/` in a feature

Handles repositories, API calls, or local storage.

```
repositories/  
  auth_repository.dart  
datasources/  
  auth_api.dart  
  auth_local_storage.dart
```

4. Advantages of Feature-Based Structure

1. **Encapsulation:** Each feature is independent.
2. **Scalability:** Easy to add new features.
3. **Team Collaboration:** Teams can work on separate features.
4. **Easier Testing:** Each feature has its own testable units.

5. Optional: Feature-Level Routing

```
features/  
  home/  
    presentation/  
      screens/home_screen.dart  
      home_router.dart
```

Example usage in `main.dart`:

```
import 'features/home/presentation/home_router.dart';  
  
void main() {  
  runApp(MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      initialRoute: '/',  
      routes: HomeRouter.routes,  
    );  
  }  
}
```

```
}  
}
```

This structure ensures **modular, maintainable, and scalable Flutter projects**, ideal for large-scale apps with multiple features.